Final Term Project

OBJECT-ORIENTED PROGRAMMING

Student names and ID:-

- **Hasan Ali (F25BDATS1M02078)**
- **Fatima Manzoor (F25BDATS1M02087)**
- **Amna Ejaz (F25BDATS1M02085)**

Chosen Library :-

- **The Matplotlib Library**

Submission Date :-

- **May 28, 2026**

OOP Analysis of the matplotlib Library

Section 1 — Library Overview

Matplotlib is Python's most popular and widely used Library. It is used for 2D visualization of objects. It was created in 2003 by John D. Hunter. The library is used in many fields like scientific research, machine learning, financial analysis, engineering, and academia etc. Major companies like NASA rely on matplotlib for making their graphs and charts.

What matplotlib Produces

- Line charts
- Bar charts
- Scatter plots
- Histograms
- Pie charts
- Heatmaps

Installation

To install we need to run this command in terminal :-

```
pip install matplotlib
```

Two Interfaces

Matplotlib provides two distinct ways to create charts.

pyplot interface

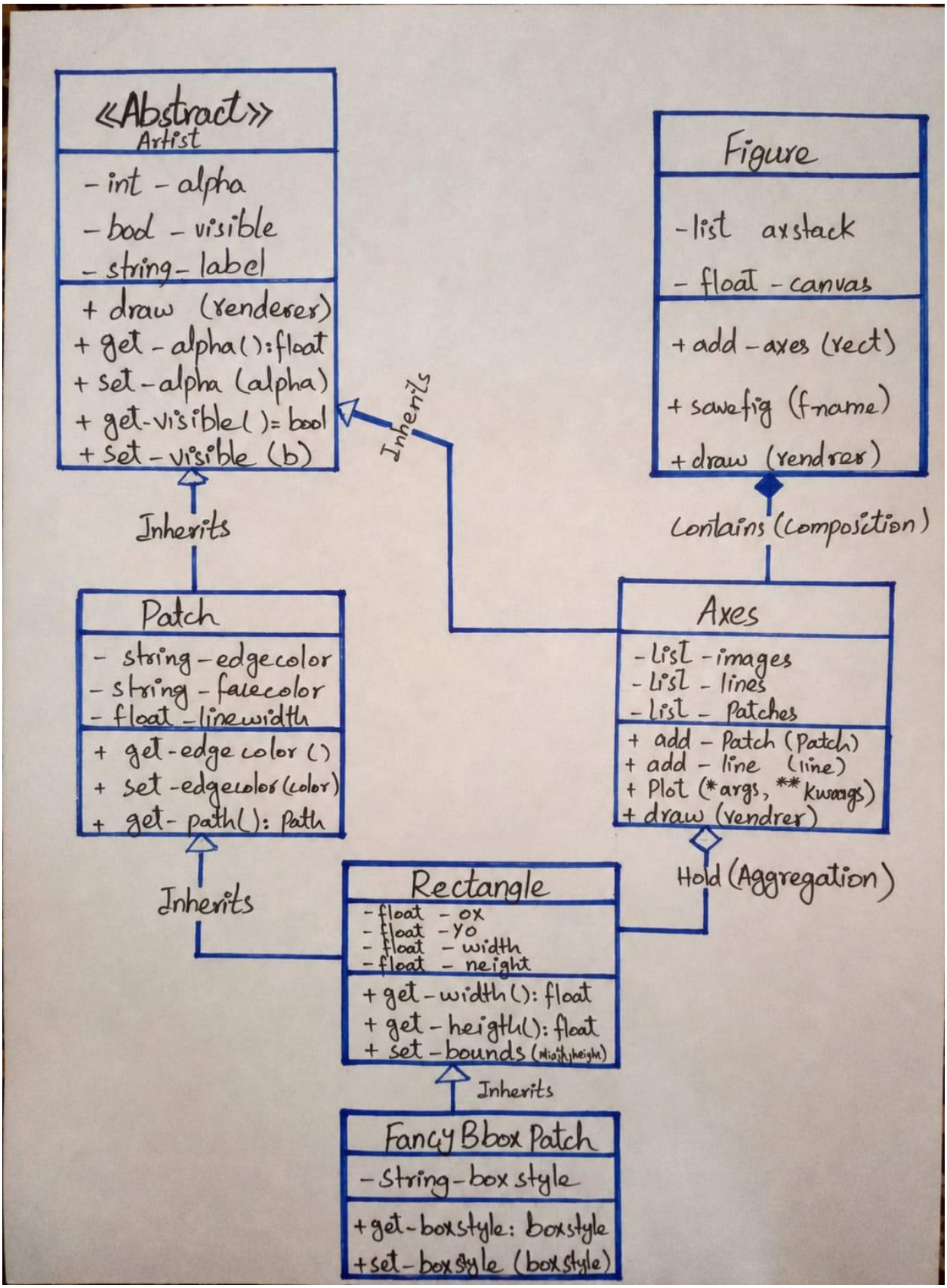
- For Quick plots, beginners, scripts

OOP interface

- For Professional use, this assignment, complex charts

Section 2 — Class Hierarchy Diagram

Matplotlib library is built on a single base class called Artist. Every visible element in a matplotlib figure, the window, the chart area, lines, shapes, labels, tick mark is a subclass of Artist. All other classes inherits from Artist class in the whole library.


«Abstract»
Artist
- int - alpha
- bool - visible
- string - label
+ draw (renderer)
+ get - alpha(): float
+ set - alpha (alpha)
+ get - visible( ) = bool
+ set - visible (b)
Figure
- list axstack
- float - canvas
+ add - axes (rect)
+ savefig (fname)
+ draw (rendrer)
Inherits
Inherits
Contains (Composition)
Patch
- string - edgecolor
- string - facecolor
- float - linewidth
+ get - edge color ( )
+ set - edgecolor (color)
+ get - path( ): Path
Axes
- List - images
- List - lines
- List - Patches
+ add - Patch (Patch)
+ add - line (line)
+ Plot (*args, ** kwargs)
+ draw (rendrer)
Inherits
Hold (Aggregation)
Rectangle
- float - ox
- float - yo
- float - width
- float - neight
+ get - width (): float
+ get - heigth(): float
+ set - bounds (width, height)
Inherits
Fancy Bbox Patch
- String - box style
+ get - boxstyle: boxstyle
+ set - box style (boxstyle)

This is a UML diagram represent relation between some classes od matplotlib.

Key Class Descriptions

`Artist:` Abstract base for all visual elements
`Figure:` Top-level window/canvas container
`Patch:` Base for Rectangle, Circle, FancyArrow
`Axes:` One chart area inside a Figure

Section 3 — OOP Principles Analysis

Now we gonna Find OOP principles from different source codes of different classes from matplotlib library. We are going to cover all 4 of OOP principles.

1 Inheritance:

Inheritance is the most prominent OOP feature in matplotlib. Every visible element in the library Figure, Axes, Line2D, Text, Rectangle, Circle inherits from the Artist base class. Here are some example of it

```
from collections import namedtuple
import contextlib
from functools import cache, reduce, wraps
import inspect
from inspect import Signature, Parameter
import logging
from numbers import Number, Real
import operator
import re
import warnings

import numpy as np

import matplotlib as mpl
from . import _api, cbook
from .path import Path
from .transforms import (BboxBase, Bbox, IdentityTransform, Transform, TransformedBbox,
                         TransformedPatchPath, TransformedPath)

_log = logging.getLogger(__name__)
```

In this code we can see this class is inheriting many methods or many other classes by using import. Inheritance is most commonly used in matplotlib library to get data of one class in another class.

2 Encapsulation:

Encapsulation is also a important and widely used principle of OOP. Encapsulation means hiding or protecting anything. It is basically used to hide complex backend data from users or for protecting data inside a class. In matplotlib's Figure class, the internal list of axes is stored as a private attribute.

```
121         artists to the figure or subfigure, create Axes, etc.
122         """
123         def __init__(self, **kwargs):
124             super().__init__()
125             # remove the non-figure artist _axes property
126             # as it makes no sense for a figure to be _in_ an Axes
127             # this is used by the property methods in the artist base class
128             # which are over-ridden in this class
129             del self._axes
130
131             self._suptitle = None
132             self._supxlabel = None
133             self._supylabel = None
134
135             # groupers to keep track of x, y labels and title we want to align.
136             # see self.align_xlabels, self.align_ylabels,
137             # self.align_titles, and axis._get_tick_boxes_siblings
138             self._align_label_groups = {
139                 "x": cbook.Grouper(),
140                 "y": cbook.Grouper(),
141                 "title": cbook.Grouper()
142             }
143
144             self._localaxes = []  # track all Axes
145             self.artists = []
146             self.lines = []
147             self.patches = []
148             self.texts = []
149             self.images = []
```

In the given image we can see use of encapsulation by identifying protected methods using underscore.

3 Polymorphism:

Polymorphism means the same method name behaves differently depending on which object it is called on. In matplotlib, every single Artist subclass has a draw method. The rendering engine simply calls draw() on each object, it does not need to know or check whether it is a Figure, Axes, Line2D, or any other type. Each one draws itself differently. This thing uses polymorphism means many shapes of a thing.

```
153                 self.suppressComposite = None
154                 self.set(**kwargs)
155
156             def _get_draw_artists(self, renderer):
157                 """Also runs apply_aspect"""
158                 artists = self.get_children()
159
396                 'ha': 'center', 'va': 'top', 'rotation': 0,
397                 'size': 'figure.titlesize', 'weight': 'figure.titleweight'}
398             return self._suplabels(t, info, **kwargs)
399
400         def get_suptitle(self):
401             """Return the suptitle as string or an empty string if not set."""
402             text_obj = self._suptitle
403             return "" if text_obj is None else text_obj.get_text()
404
405         @_docstring.Substitution(x0=0.5, y0=0.01, name='super xlabel', ha='center',
406                                  va='bottom', rc='label')
407         @_docstring.copy(_suplabels)
```

Here we can see some use of polymorphism in matplotlib.

4 Abstraction:

Abstraction means hiding complex internal implementation behind a simple interface. When you call ax.plot(), you are asking for a line to be drawn. Behind the scenes, matplotlib creates a Line2D object, converts your data coordinates to pixel coordinates, registers it with the axes, manages z-ordering, handles color cycles, and dozens of other operations. None of this complexity is exposed to the user.

```
112         def __setstate__(self, state):
113             next_counter = state.pop('_counter')
114             vars(self).update(state)
115             self._counter = itertools.count(next_counter)
116
117
118     class FigureBase(Artist):
119         """
120         Base class for `.Figure` and `.SubFigure` containing the methods that add
121         artists to the figure or subfigure, create Axes, etc.
122         """
123         def __init__(self, **kwargs):
```

```
503         def add_artist(self, artist, clip=False):
517             clip : bool, default: False
518                 Whether the added artist should be clipped by the figure patc
519
520             Returns
521             -------
522             `~matplotlib.artist.Artist`
523                 The added artist.
524             """
525             artist.set_figure(self)
526             self.artists.append(artist)
527             artist._remove_method = self.artists.remove
528
```

Section 4 — Design Decision Critique

The most noticeable design in matplotlib library is every single class inherits from a single class Artist. Because of this, Figure, Axes, Line, Text, Rectangle, and all other visual objects follow the same system. The Artist class provides common features such as draw(), alpha, visible, and zorder.

This design has several advantages. The rendering system can handle every object in the same way by simply calling the draw() method without checking the object type. It also provides a consistent API because the same methods like set_visible() and set_alpha() work on all visual elements. In addition, the system is extensible because developers can create new visual elements by inheriting from Artist.

The only bad thing here is because Artist class holds all data it become a large class. This violates the Single Responsibility Principle, which states that a class should have only one responsibility. The deep inheritance hierarchy also makes the code difficult to understand because methods may come from multiple parent classes. Also the entire library depends

heavily on Artist so any change in the Artist class can affect almost every other class in matplotlib.
A more modern alternative would be to use a Protocol instead of inheritance. In this approach, any class that implements a draw() method would automatically work with the rendering system without needing to inherit from Artist.

Section 5 — Custom Extension Code